📚 ED JAVA DOC - Documentación JavaDoc Completa

Introducción

Este documento proporciona la documentación JavaDoc completa de todas las clases principales de SportConnect, incluyendo métodos, parámetros, retornos y ejemplos de uso.

---

1. Clase User

JavaDoc Completo

Java

```java
/**
 * Clase que representa un usuario de la aplicación SportConnect.
 *
 * Un usuario puede ser un entrenador o un atleta. Los entrenadores pueden
 * crear actividades deportivas, mientras que los atletas pueden apuntarse
 * a las actividades disponibles.
 *
 * @author SportConnect Team
 * @version 1.0
 * @since 2026-04-28
 *
 * @see Activity
 * @see ActivityParticipant
 */
public class User {

    /**
     * Identificador único del usuario en la base de datos.
     * Se genera automáticamente al crear el usuario.
     */
    private int id;

    /**
     * Nombre completo del usuario.
     * No puede ser nulo ni vacío.
     * Mínimo 3 caracteres.
     */
    private String name;
```

```java
    /**
     * Email del usuario.
     * Debe ser único en la base de datos.
     * Debe cumplir formato válido de email.
     */
    private String email;

    /**
     * Contraseña del usuario (almacenada con hash).
     * Mínimo 6 caracteres.
     * Se almacena hasheada por seguridad.
     */
    private String password;

    /**
     * Rol del usuario en la aplicación.
     * Valores válidos: "trainer" (entrenador) o "athlete" (atleta).
     * Determina qué funcionalidades puede usar el usuario.
     */
    private String role;

    /**
     * Deporte favorito del usuario.
     * Se usa para filtrar actividades relevantes.
     * Ejemplos: "Fútbol", "Yoga", "Natación", etc.
     */
    private String sport;

    /**
     * Fecha de creación del usuario.
     * Se establece automáticamente al registrarse.
     */
    private Date createdAt;

    /**
     * Fecha de última actualización del perfil.
     * Se actualiza cada vez que se modifica el usuario.
     */
    private Date updatedAt;

    /**
     * Constructor vacío de la clase User.
     * Necesario para la serialización y creación de objetos vacíos.
     */
    public User() {}

    /**
     * Constructor con parámetros principales.
```

```java
     *
     * @param name Nombre del usuario (mínimo 3 caracteres)
     * @param email Email del usuario (debe ser válido)
     * @param password Contraseña del usuario (mínimo 6 caracteres)
     * @param role Rol del usuario ("trainer" o "athlete")
     * @param sport Deporte favorito del usuario
     *
     * @throws IllegalArgumentException si algún parámetro es inválido
     *
     * @example
     * User user = new User("Juan García", "juan@example.com",
     *                      "password123", "trainer", "Yoga");
     */
    public User(String name, String email, String password,
                String role, String sport) {
        this.name = name;
        this.email = email;
        this.password = password;
        this.role = role;
        this.sport = sport;
        this.createdAt = new Date();
        this.updatedAt = new Date();
    }

    /**
     * Obtiene el identificador único del usuario.
     *
     * @return El ID del usuario
     */
    public int getId() {
        return id;
    }

    /**
     * Establece el identificador único del usuario.
     *
     * @param id El ID a establecer
     */
    public void setId(int id) {
        this.id = id;
    }

    /**
     * Obtiene el nombre del usuario.
     *
     * @return El nombre del usuario
     */
    public String getName() {
```

```java
        return name;
    }

    /**
     * Establece el nombre del usuario.
     *
     * @param name El nombre a establecer (mínimo 3 caracteres)
     * @throws IllegalArgumentException si el nombre es muy corto
     */
    public void setName(String name) {
        if (name != null && name.length() >= 3) {
            this.name = name;
            this.updatedAt = new Date();
        } else {
            throw new IllegalArgumentException("El nombre debe tener al menos 3
        }
    }

    /**
     * Obtiene el email del usuario.
     *
     * @return El email del usuario
     */
    public String getEmail() {
        return email;
    }

    /**
     * Establece el email del usuario.
     *
     * @param email El email a establecer
     * @throws IllegalArgumentException si el email no es válido
     */
    public void setEmail(String email) {
        if (validateEmail(email)) {
            this.email = email;
            this.updatedAt = new Date();
        } else {
            throw new IllegalArgumentException("Email no válido");
        }
    }

    /**
     * Obtiene la contraseña del usuario (hasheada).
     *
     * @return La contraseña hasheada
     *
     * @note La contraseña siempre se devuelve hasheada por seguridad.
```

```java
     *       Nunca se devuelve en texto plano.
     */
    public String getPassword() {
        return password;
    }

    /**
     * Establece la contraseña del usuario.
     * La contraseña se hashea antes de almacenarse.
     *
     * @param password La contraseña a establecer
     * @throws IllegalArgumentException si la contraseña es muy corta
     */
    public void setPassword(String password) {
        if (validatePassword(password)) {
            // Aquí se haría el hash de la contraseña
            this.password = password;
            this.updatedAt = new Date();
        } else {
            throw new IllegalArgumentException("La contraseña debe tener al men
        }
    }

    /**
     * Obtiene el rol del usuario.
     *
     * @return El rol ("trainer" o "athlete")
     */
    public String getRole() {
        return role;
    }

    /**
     * Establece el rol del usuario.
     *
     * @param role El rol a establecer ("trainer" o "athlete")
     * @throws IllegalArgumentException si el rol no es válido
     */
    public void setRole(String role) {
        if ("trainer".equalsIgnoreCase(role) || "athlete".equalsIgnoreCase(role
            this.role = role;
            this.updatedAt = new Date();
        } else {
            throw new IllegalArgumentException("Rol no válido. Use 'trainer' o
        }
    }

    /**
```

```java
     * Obtiene el deporte favorito del usuario.
     *
     * @return El deporte favorito
     */
    public String getSport() {
        return sport;
    }

    /**
     * Establece el deporte favorito del usuario.
     *
     * @param sport El deporte a establecer
     */
    public void setSport(String sport) {
        this.sport = sport;
        this.updatedAt = new Date();
    }

    /**
     * Verifica si el usuario es un entrenador.
     *
     * @return true si el usuario es entrenador, false en caso contrario
     *
     * @example
     * if (user.isTrainer()) {
     *     // Mostrar opciones de entrenador
     * }
     */
    public boolean isTrainer() {
        return "trainer".equalsIgnoreCase(role);
    }

    /**
     * Verifica si el usuario es un atleta.
     *
     * @return true si el usuario es atleta, false en caso contrario
     *
     * @example
     * if (user.isAthlete()) {
     *     // Mostrar opciones de atleta
     * }
     */
    public boolean isAthlete() {
        return "athlete".equalsIgnoreCase(role);
    }

    /**
     * Valida el formato del email.
```

```java
     *
     * @param email El email a validar
     * @return true si el email es válido, false en caso contrario
     *
     * @note La validación es básica (solo verifica que contenga @).
     *       Para validación completa, usar expresión regular.
     *
     * @example
     * if (User.validateEmail("juan@example.com")) {
     *     // Email válido
     * }
     */
    public static boolean validateEmail(String email) {
        return email != null && email.contains("@") && email.contains(".");
    }

    /**
     * Valida la contraseña.
     *
     * @param password La contraseña a validar
     * @return true si la contraseña es válida, false en caso contrario
     *
     * @note La contraseña debe tener al menos 6 caracteres.
     */
    public static boolean validatePassword(String password) {
        return password != null && password.length() >= 6;
    }

    /**
     * Convierte el objeto User a una representación en String.
     *
     * @return String con la información del usuario
     *
     * @example
     * User user = new User(...);
     * System.out.println(user);
     * // Output: User{id=1, name='Juan García', email='juan@example.com', role
     */
    @Override
    public String toString() {
        return "User{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", email='" + email + '\'' +
                ", role='" + role + '\'' +
                ", sport='" + sport + '\'' +
                '}';
```

```
    }
}
```

---

2. Clase Activity

JavaDoc Completo

Java

```java
/**
 * Clase que representa una actividad deportiva en SportConnect.
 *
 * Una actividad es creada por un entrenador y puede tener múltiples
 * participantes (atletas). Contiene información sobre el deporte,
 * fecha, hora, ubicación y capacidad máxima.
 *
 * @author SportConnect Team
 * @version 1.0
 * @since 2026-04-28
 *
 * @see User
 * @see ActivityParticipant
 */
public class Activity {

    /**
     * Identificador único de la actividad.
     * Se genera automáticamente al crear la actividad.
     */
    private int id;

    /**
     * Título o nombre de la actividad.
     * Ejemplo: "Yoga Matutino", "Fútbol 5", "Natación".
     */
    private String title;

    /**
     * Tipo de deporte de la actividad.
     * Ejemplos: "Yoga", "Fútbol", "Natación", "Tenis".
     */
    private String sport;

    /**
     * Descripción detallada de la actividad.
```

```java
     * Puede incluir información adicional, requisitos, etc.
     */
    private String description;

    /**
     * Fecha en la que se realiza la actividad.
     * Formato: yyyy-MM-dd
     */
    private Date date;

    /**
     * Hora de inicio de la actividad.
     * Formato: HH:mm
     */
    private Time time;

    /**
     * Ubicación donde se realiza la actividad.
     * Ejemplo: "Centro Deportivo Municipal", "Calle Principal 123".
     */
    private String location;

    /**
     * Número máximo de participantes permitidos.
     * Una vez alcanzado, no se pueden añadir más participantes.
     */
    private int maxParticipants;

    /**
     * ID del entrenador que creó la actividad.
     * Referencia a la tabla users.
     */
    private int trainerId;

    /**
     * Fecha de creación de la actividad.
     * Se establece automáticamente.
     */
    private Date createdAt;

    /**
     * Fecha de última actualización de la actividad.
     * Se actualiza cada vez que se modifica.
     */
    private Date updatedAt;

    /**
     * Constructor vacío de la clase Activity.
```

```java
     */
    public Activity() {}

    /**
     * Constructor con parámetros principales.
     *
     * @param title Título de la actividad
     * @param sport Tipo de deporte
     * @param description Descripción de la actividad
     * @param date Fecha de la actividad
     * @param time Hora de la actividad
     * @param location Ubicación de la actividad
     * @param maxParticipants Máximo de participantes
     * @param trainerId ID del entrenador
     *
     * @example
     * Activity activity = new Activity(
     *     "Yoga Matutino",
     *     "Yoga",
     *     "Sesión de yoga relajante",
     *     new Date(2026, 4, 28),
     *     new Time(8, 0),
     *     "Centro Deportivo",
     *     15,
     *     1
     * );
     */
    public Activity(String title, String sport, String description,
                    Date date, Time time, String location,
                    int maxParticipants, int trainerId) {
        this.title = title;
        this.sport = sport;
        this.description = description;
        this.date = date;
        this.time = time;
        this.location = location;
        this.maxParticipants = maxParticipants;
        this.trainerId = trainerId;
        this.createdAt = new Date();
        this.updatedAt = new Date();
    }

    // Getters y Setters (similar a User)

    /**
     * Verifica si la actividad está llena.
     *
     * @return true si se alcanzó el máximo de participantes, false en caso con
```

```java
     *
     * @example
     * if (activity.isFull()) {
     *     // No se pueden añadir más participantes
     * }
     */
    public boolean isFull() {
        return getParticipantCount() >= maxParticipants;
    }

    /**
     * Obtiene el número de lugares disponibles.
     *
     * @return Número de lugares disponibles
     *
     * @example
     * int available = activity.hasAvailableSpots();
     * System.out.println("Lugares disponibles: " + available);
     */
    public int hasAvailableSpots() {
        return maxParticipants - getParticipantCount();
    }

    /**
     * Obtiene el número actual de participantes.
     *
     * @return Número de participantes inscritos
     *
     * @note Este método consulta la base de datos.
     */
    public int getParticipantCount() {
        // Implementación: consultar BD
        return 0;
    }

    /**
     * Convierte el objeto Activity a una representación en String.
     *
     * @return String con la información de la actividad
     */
    @Override
    public String toString() {
        return "Activity{" +
                "id=" + id +
                ", title='" + title + '\'' +
                ", sport='" + sport + '\'' +
                ", date=" + date +
                ", location='" + location + '\'' +
```

```
                ", maxParticipants=" + maxParticipants +
                '}';
    }
}
```

3. Clase DatabaseHelper

JavaDoc Completo

Java

```java
/**
 * Clase que gestiona la base de datos SQLite de SportConnect.
 *
 * DatabaseHelper es responsable de:
 * - Crear y actualizar el esquema de la BD
 * - Realizar operaciones CRUD en todas las tablas
 * - Manejar transacciones
 * - Cerrar conexiones correctamente
 *
 * Implementa el patrón Singleton para garantizar una única instancia.
 *
 * @author SportConnect Team
 * @version 1.0
 * @since 2026-04-28
 *
 * @see User
 * @see Activity
 * @see ActivityParticipant
 */
public class DatabaseHelper extends SQLiteOpenHelper {

    /**
     * Nombre de la base de datos.
     */
    private static final String DB_NAME = "sportconnect.db";

    /**
     * Versión de la base de datos.
     * Incrementar cuando se modifica el esquema.
     */
    private static final int DB_VERSION = 1;

    /**
     * Nombre de la tabla de usuarios.
```

```java
     */
    private static final String TABLE_USERS = "users";

    /**
     * Nombre de la tabla de actividades.
     */
    private static final String TABLE_ACTIVITIES = "activities";

    /**
     * Nombre de la tabla de participantes.
     */
    private static final String TABLE_PARTICIPANTS = "participants";

    // Columnas de la tabla USERS
    private static final String COL_USER_ID = "id";
    private static final String COL_USER_NAME = "name";
    private static final String COL_USER_EMAIL = "email";
    private static final String COL_USER_PASSWORD = "password";
    private static final String COL_USER_ROLE = "role";
    private static final String COL_USER_SPORT = "sport";

    // Columnas de la tabla ACTIVITIES
    private static final String COL_ACT_ID = "id";
    private static final String COL_ACT_TITLE = "title";
    private static final String COL_ACT_SPORT = "sport";
    private static final String COL_ACT_DATE = "date";
    private static final String COL_ACT_TIME = "time";
    private static final String COL_ACT_LOCATION = "location";
    private static final String COL_ACT_MAX_PART = "max_participants";
    private static final String COL_ACT_TRAINER_ID = "trainer_id";

    // Columnas de la tabla PARTICIPANTS
    private static final String COL_PART_USER_ID = "user_id";
    private static final String COL_PART_ACT_ID = "activity_id";
    private static final String COL_PART_JOIN_DATE = "join_date";

    /**
     * Constructor de DatabaseHelper.
     *
     * @param context El contexto de la aplicación
     *
     * @example
     * DatabaseHelper dbHelper = new DatabaseHelper(context);
     */
    public DatabaseHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }
```

```java
    /**
     * Se llama cuando se crea la base de datos por primera vez.
     * Crea todas las tablas necesarias.
     *
     * @param db La base de datos
     *
     * @note Este método se ejecuta automáticamente la primera vez.
     */
    @Override
    public void onCreate(SQLiteDatabase db) {
        // Crear tablas
        String createUsersTable = "CREATE TABLE " + TABLE_USERS + " (" +
                COL_USER_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
                COL_USER_NAME + " TEXT NOT NULL, " +
                COL_USER_EMAIL + " TEXT UNIQUE NOT NULL, " +
                COL_USER_PASSWORD + " TEXT NOT NULL, " +
                COL_USER_ROLE + " TEXT NOT NULL, " +
                COL_USER_SPORT + " TEXT" +
                ")";

        db.execSQL(createUsersTable);

        // Crear más tablas...
    }

    /**
     * Se llama cuando se actualiza la versión de la BD.
     * Modifica el esquema según sea necesario.
     *
     * @param db La base de datos
     * @param oldVersion La versión anterior
     * @param newVersion La nueva versión
     */
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        // Actualizar esquema
        if (oldVersion < 2) {
            // Cambios para versión 2
        }
    }

    /**
     * Obtiene un usuario por su email.
     *
     * @param email El email del usuario a buscar
     * @return El objeto User si existe, null si no
     *
     * @throws SQLException si hay error en la BD
```

```
     *
     * @example
     * User user = dbHelper.getUserByEmail("juan@example.com");
     * if (user != null) {
     *     // Usuario encontrado
     * }
     */
    public User getUserByEmail(String email) {
        SQLiteDatabase db = this.getReadableDatabase();
        Cursor cursor = db.query(
            TABLE_USERS,
            null,
            COL_USER_EMAIL + " = ?",
            new String[]{email},
            null,
            null,
            null
        );

        User user = null;
        if (cursor.moveToFirst()) {
            user = new User();
            user.setId(cursor.getInt(0));
            user.setName(cursor.getString(1));
            user.setEmail(cursor.getString(2));
            // ... más campos
        }
        cursor.close();
        return user;
    }

    /**
     * Crea un nuevo usuario en la BD.
     *
     * @param user El objeto User a crear
     * @return El ID del usuario creado, -1 si hay error
     *
     * @throws SQLException si hay error en la BD
     *
     * @example
     * User newUser = new User("Juan", "juan@example.com", "pass123", "trainer",
     * long userId = dbHelper.createUser(newUser);
     * if (userId > 0) {
     *     // Usuario creado exitosamente
     * }
     */
    public long createUser(User user) {
        SQLiteDatabase db = this.getWritableDatabase();
```

```java
        ContentValues values = new ContentValues();
        values.put(COL_USER_NAME, user.getName());
        values.put(COL_USER_EMAIL, user.getEmail());
        values.put(COL_USER_PASSWORD, user.getPassword());
        values.put(COL_USER_ROLE, user.getRole());
        values.put(COL_USER_SPORT, user.getSport());

        return db.insert(TABLE_USERS, null, values);
    }

    /**
     * Actualiza un usuario existente.
     *
     * @param user El objeto User con los datos actualizados
     * @return El número de filas actualizadas
     *
     * @example
     * user.setName("Juan García");
     * int updated = dbHelper.updateUser(user);
     */
    public int updateUser(User user) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COL_USER_NAME, user.getName());
        values.put(COL_USER_EMAIL, user.getEmail());
        // ... más campos

        return db.update(TABLE_USERS, values, COL_USER_ID + " = ?",
                new String[]{String.valueOf(user.getId())});
    }

    /**
     * Elimina un usuario de la BD.
     *
     * @param userId El ID del usuario a eliminar
     * @return El número de filas eliminadas
     *
     * @example
     * int deleted = dbHelper.deleteUser(1);
     */
    public int deleteUser(int userId) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_USERS, COL_USER_ID + " = ?",
                new String[]{String.valueOf(userId)});
    }

    /**
     * Obtiene todas las actividades de la BD.
```

```java
     *
     * @return Lista de todas las actividades
     *
     * @example
     * List<Activity> activities = dbHelper.getAllActivities();
     * for (Activity activity : activities) {
     *     System.out.println(activity.getTitle());
     * }
     */
    public List<Activity> getAllActivities() {
        List<Activity> activities = new ArrayList<>();
        SQLiteDatabase db = this.getReadableDatabase();
        Cursor cursor = db.query(TABLE_ACTIVITIES, null, null, null, null, null

        if (cursor.moveToFirst()) {
            do {
                Activity activity = new Activity();
                activity.setId(cursor.getInt(0));
                activity.setTitle(cursor.getString(1));
                // ... más campos
                activities.add(activity);
            } while (cursor.moveToNext());
        }
        cursor.close();
        return activities;
    }

    /**
     * Obtiene una actividad por su ID.
     *
     * @param id El ID de la actividad
     * @return El objeto Activity si existe, null si no
     *
     * @example
     * Activity activity = dbHelper.getActivityById(1);
     */
    public Activity getActivityById(int id) {
        // Implementación similar a getUserByEmail
        return null;
    }

    /**
     * Crea una nueva actividad en la BD.
     *
     * @param activity El objeto Activity a crear
     * @return El ID de la actividad creada, -1 si hay error
     *
     * @example
```

```java
     * Activity newActivity = new Activity(...);
     * long activityId = dbHelper.createActivity(newActivity);
     */
    public long createActivity(Activity activity) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COL_ACT_TITLE, activity.getTitle());
        values.put(COL_ACT_SPORT, activity.getSport());
        // ... más campos

        return db.insert(TABLE_ACTIVITIES, null, values);
    }

    /**
     * Actualiza una actividad existente.
     *
     * @param activity El objeto Activity con los datos actualizados
     * @return El número de filas actualizadas
     */
    public int updateActivity(Activity activity) {
        // Implementación
        return 0;
    }

    /**
     * Elimina una actividad de la BD.
     *
     * @param id El ID de la actividad a eliminar
     * @return El número de filas eliminadas
     */
    public int deleteActivity(int id) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_ACTIVITIES, COL_ACT_ID + " = ?",
                new String[]{String.valueOf(id)});
    }

    /**
     * Añade un participante a una actividad.
     *
     * @param userId El ID del usuario
     * @param activityId El ID de la actividad
     * @return El ID del registro creado, -1 si hay error
     *
     * @example
     * long result = dbHelper.addParticipant(1, 5);
     */
    public long addParticipant(int userId, int activityId) {
        SQLiteDatabase db = this.getWritableDatabase();
```

```java
        ContentValues values = new ContentValues();
        values.put(COL_PART_USER_ID, userId);
        values.put(COL_PART_ACT_ID, activityId);
        values.put(COL_PART_JOIN_DATE, new Date().getTime());

        return db.insert(TABLE_PARTICIPANTS, null, values);
    }

    /**
     * Elimina un participante de una actividad.
     *
     * @param userId El ID del usuario
     * @param activityId El ID de la actividad
     * @return El número de filas eliminadas
     *
     * @example
     * int removed = dbHelper.removeParticipant(1, 5);
     */
    public int removeParticipant(int userId, int activityId) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_PARTICIPANTS,
                COL_PART_USER_ID + " = ? AND " + COL_PART_ACT_ID + " = ?",
                new String[]{String.valueOf(userId), String.valueOf(activityId)
    }

    /**
     * Obtiene el número de participantes en una actividad.
     *
     * @param activityId El ID de la actividad
     * @return El número de participantes
     *
     * @example
     * int count = dbHelper.getParticipantCount(5);
     * System.out.println("Participantes: " + count);
     */
    public int getParticipantCount(int activityId) {
        SQLiteDatabase db = this.getReadableDatabase();
        Cursor cursor = db.rawQuery(
            "SELECT COUNT(*) FROM " + TABLE_PARTICIPANTS +
            " WHERE " + COL_PART_ACT_ID + " = ?",
            new String[]{String.valueOf(activityId)}
        );

        int count = 0;
        if (cursor.moveToFirst()) {
            count = cursor.getInt(0);
        }
        cursor.close();
```

```
        return count;
    }

    /**
     * Cierra la conexión con la base de datos.
     * Debe llamarse cuando se termina de usar la BD.
     *
     * @example
     * dbHelper.close();
     */
    @Override
    public void close() {
        super.close();
    }
}
```

---

4. Mejores Prácticas de JavaDoc

Estructura Recomendada

Java

```
/**
 * Descripción breve de la clase/método (una línea).
 *
 * Descripción detallada (puede ocupar varias líneas).
 * Explicar qué hace, cómo funciona, casos especiales.
 *
 * @author Nombre del autor
 * @version Versión
 * @since Fecha de creación
 *
 * @param nombreParametro Descripción del parámetro
 * @return Descripción del valor retornado
 * @throws TipoExcepcion Cuándo se lanza esta excepción
 *
 * @see ClaseRelacionada
 * @see OtraClase
 *
 * @example
 * // Ejemplo de uso
 * MiClase objeto = new MiClase();
 * objeto.miMetodo();
 */
```

Tags Comunes

Tag	Uso
`@author`	Autor del código
`@version`	Versión del código
`@since`	Desde qué versión existe
`@param`	Descripción de parámetro
`@return`	Descripción del retorno
`@throws`	Excepciones que puede lanzar
`@see`	Referencias a otras clases
`@example`	Ejemplo de uso
`@deprecated`	Marca como obsoleto
`@note`	Nota importante

---

5. Generación de Documentación

Comando Javadoc

Bash

```bash
# Generar documentación para un archivo
javadoc User.java

# Generar para múltiples archivos
javadoc *.java

# Generar con opciones
javadoc -d docs -private -author -version *.java
```

Opciones Útiles

Bash

```
-d <directorio>      # Directorio de salida
-private             # Incluir miembros privados
-public              # Solo miembros públicos (default)
-author              # Incluir @author
-version             # Incluir @version
-windowtitle <título> # Título de la ventana
-doctitle <título>   # Título de la documentación
```

---

6. Ejemplo de Documentación Completa

Java

```java
package com.sportconnect.database;

import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

/**
 * Gestor de base de datos SQLite para SportConnect.
 *
 * Esta clase maneja todas las operaciones de base de datos incluyendo
 * creación de tablas, operaciones CRUD y consultas complejas.
 *
 * Características:
 * - Patrón Singleton para instancia única
 * - Manejo seguro de transacciones
 * - Validación de datos
 * - Manejo de excepciones
 *
 * @author SportConnect Team
 * @version 1.0
 * @since 2026-04-28
 *
 * @see User
 * @see Activity
 * @see ActivityParticipant
 *
 * @example
 * DatabaseHelper dbHelper = DatabaseHelper.getInstance(context);
 * User user = dbHelper.getUserByEmail("juan@example.com");
 * List<Activity> activities = dbHelper.getAllActivities();
 */
public class DatabaseHelper extends SQLiteOpenHelper {
```

```
    // ... código ...
}
```

---

7. Conclusión

La documentación JavaDoc es esencial para:

- Entender el código
- Facilitar mantenimiento
- Ayudar a otros desarrolladores
- Generar documentación automática
- Mejorar la calidad del código

¡Documenta tu código mientras lo escribes! 📚